

Interfaces – Employee Salary Details

Experiment No: 07

Date: 19/09/2026

■ Objective / Aim

To design and implement a Java program using interfaces to calculate and display salary details for different types of employees, demonstrating abstraction and multiple inheritance of type through interfaces.

■ System Requirements

Hardware

- Standard PC/Laptop
- Intel Core i3/i5 processor or equivalent
- Minimum 4 GB RAM
- Minimum 500 MB free storage

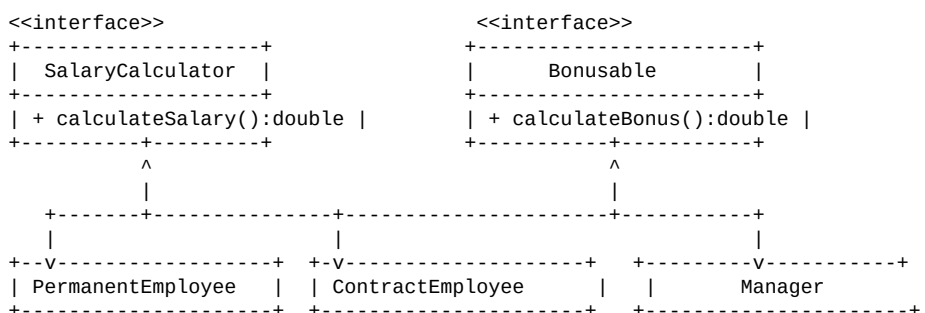
Software

- Operating System: Windows / Linux / macOS
- Java Development Kit (JDK 17 or JDK 21)
- Text Editor or IDE such as Eclipse, IntelliJ IDEA, NetBeans, or VS Code
- Command Prompt / Terminal

■ Algorithm / Logic Description

1. Define an interface **SalaryCalculator** with an abstract method `calculateSalary(): double`.
2. Define an interface **Bonusable** with an abstract method `calculateBonus(): double`.
3. Create a class **PermanentEmployee** that implements both **SalaryCalculator** and **Bonusable**, storing `basicPay` and providing salary as basic pay plus allowances, and bonus as a percentage of basic pay.
4. Create a class **ContractEmployee** that implements only **SalaryCalculator**, storing `hoursWorked` and `hourlyRate`, and calculating salary as `hoursWorked * hourlyRate` (contract employees are not eligible for bonus).
5. Create a class **Manager** that implements both **SalaryCalculator** and **Bonusable**, storing `basicPay` and `teamSize`, with a higher bonus rate that also factors in team size.
6. Define a method `displaySalarySlip()` in each class (or a common helper) to print employee name, computed salary, and bonus (if applicable).
7. In `main()`, create objects of **PermanentEmployee**, **ContractEmployee**, and **Manager**, store **SalaryCalculator** references in an array, and invoke `calculateSalary()` polymorphically.
8. Check `instanceof Bonusable` for each object before calling `calculateBonus()`, since only some classes implement it.
9. Display full salary details for each employee.

■ UML Class Diagram



- name: String		- name: String		- name: String	
- basicPay: double		- hoursWorked: double		- basicPay: double	
- allowances: double		- hourlyRate: double		- teamSize: int	
+-----+		+-----+		+-----+	
+ calculateSalary()		+ calculateSalary()		+ calculateSalary()	
+ calculateBonus()		+-----+		+ calculateBonus()	
+-----+				+-----+	

Implements: PermanentEmployee → SalaryCalculator, Bonusable | ContractEmployee → SalaryCalculator only | Manager → SalaryCalculator, Bonusable

Relationship: PermanentEmployee, ContractEmployee, and Manager all implement SalaryCalculator (multiple classes implementing one interface = abstraction + polymorphism). PermanentEmployee and Manager additionally implement Bonusable, showing that a class can implement multiple interfaces (Java's substitute for multiple inheritance), while ContractEmployee opts out of that behavior entirely.

■ Source Code

```
// Interface 1
interface SalaryCalculator {
    double calculateSalary();
}

// Interface 2
interface Bonusable {
    double calculateBonus();
}

// Permanent Employee - implements both interfaces
class PermanentEmployee implements SalaryCalculator, Bonusable {
    private String name;
    private double basicPay;
    private double allowances;

    PermanentEmployee(String name, double basicPay, double allowances) {
        this.name = name;
        this.basicPay = basicPay;
        this.allowances = allowances;
    }

    @Override
    public double calculateSalary() {
        return basicPay + allowances;
    }

    @Override
    public double calculateBonus() {
        return basicPay * 0.10; // 10% of basic pay
    }

    public String getName() {
        return name;
    }
}

// Contract Employee - implements only SalaryCalculator
class ContractEmployee implements SalaryCalculator {
    private String name;
    private double hoursWorked;
    private double hourlyRate;

    ContractEmployee(String name, double hoursWorked, double hourlyRate) {
        this.name = name;
        this.hoursWorked = hoursWorked;
        this.hourlyRate = hourlyRate;
    }

    @Override
    public double calculateSalary() {
        return hoursWorked * hourlyRate;
    }

    public String getName() {
        return name;
    }
}
```

```
// Manager - implements both interfaces, with team-size-based bonus
class Manager implements SalaryCalculator, Bonusable {
    private String name;
    private double basicPay;
    private int teamSize;

    Manager(String name, double basicPay, int teamSize) {
        this.name = name;
        this.basicPay = basicPay;
        this.teamSize = teamSize;
    }

    @Override
    public double calculateSalary() {
        return basicPay + (teamSize * 500); // team handling allowance
    }

    @Override
    public double calculateBonus() {
        return basicPay * 0.20 + (teamSize * 200); // 20% + per-member bonus
    }

    public String getName() {
        return name;
    }
}

// Main class
public class EmployeeSalaryDemo {
    public static void main(String[] args) {

        SalaryCalculator[] employees = {
            new PermanentEmployee("Ananya", 40000, 5000),
            new ContractEmployee("Rahul", 160, 350),
            new Manager("Karthik", 60000, 8)
        };

        for (SalaryCalculator emp : employees) {
            String name = getName(emp);
            double salary = emp.calculateSalary();

            System.out.println("==== " + name + " (" + emp.getClass().getSimpleName() + ") =====");
            System.out.printf("Salary : Rs.%.2f\n", salary);

            if (emp instanceof Bonusable) {
                double bonus = ((Bonusable) emp).calculateBonus();
                System.out.printf("Bonus : Rs.%.2f\n", bonus);
                System.out.printf("Total : Rs.%.2f\n", salary + bonus);
            } else {
                System.out.println("Bonus : Not applicable (contract employee)");
            }
            System.out.println();
        }

        // Helper to fetch name regardless of concrete type
        private static String getName(SalaryCalculator emp) {
            if (emp instanceof PermanentEmployee pe) return pe.getName();
            if (emp instanceof ContractEmployee ce) return ce.getName();
            if (emp instanceof Manager m) return m.getName();
            return "Unknown";
        }
    }
}
```

■ Compilation & Execution Commands

Save the source code exactly as:

EmployeeSalaryDemo.java

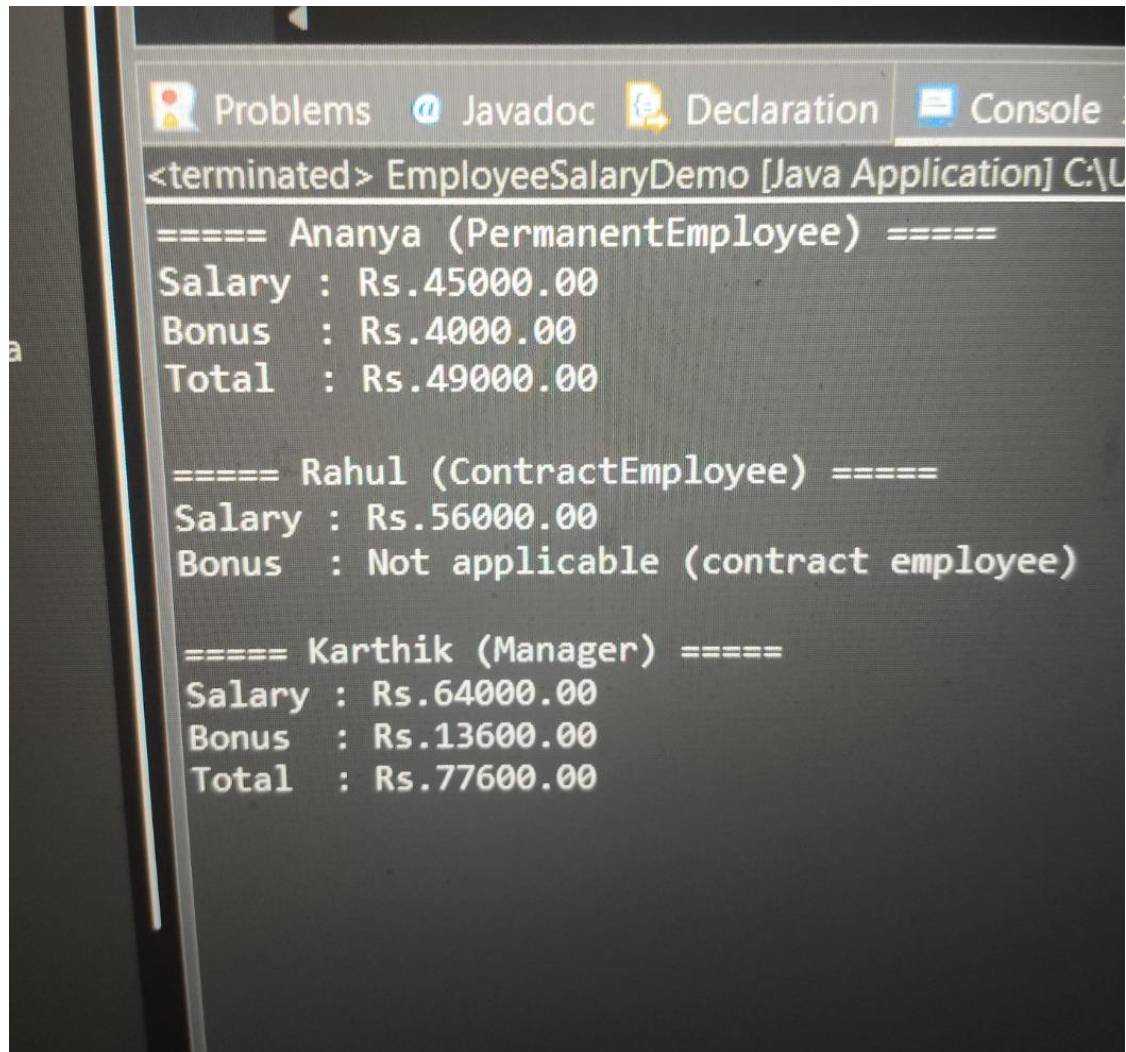
Compile the program:

javac EmployeeSalaryDemo.java

Run the compiled program:

java EmployeeSalaryDemo

■ Output Screen



```
<terminated> EmployeeSalaryDemo [Java Application] C:\U
===== Ananya (PermanentEmployee) =====
Salary : Rs.45000.00
Bonus  : Rs.4000.00
Total  : Rs.49000.00

===== Rahul (ContractEmployee) =====
Salary : Rs.56000.00
Bonus  : Not applicable (contract employee)

===== Karthik (Manager) =====
Salary : Rs.64000.00
Bonus  : Rs.13600.00
Total  : Rs.77600.00
```

Program Output (text):

```
===== Ananya (PermanentEmployee) =====
Salary : Rs.45000.00
Bonus  : Rs.4000.00
Total  : Rs.49000.00

===== Rahul (ContractEmployee) =====
Salary : Rs.56000.00
Bonus  : Not applicable (contract employee)

===== Karthik (Manager) =====
Salary : Rs.64000.00
Bonus  : Rs.13600.00
Total  : Rs.77600.00
```

■ Result

The Employee Salary Details program was successfully implemented in Java using interfaces. The program demonstrates abstraction through SalaryCalculator and Bonusable, and shows how a class can implement multiple interfaces (PermanentEmployee, Manager) or just one (ContractEmployee), enabling flexible, polymorphic salary and bonus computation.